TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH

Môn học: Hệ điều hành thời gian thực (RTOS)

BÁO CÁO PROJECT CÁ NHÂN

Hệ thống đa nhiệm FreeRTOS trên ESP32-S3: LED, nút nhấn, cảm biến và OLED đồng bộ bằng semaphore, mutex, queue và event group

Giảng viên hướng dẫn: Huỳnh Hoàng Hà

Sinh viên thực hiện: **Lê Ngọc Hoàn** MSSV: **24119038**

Mã nguồn và video: https://github.com/JackGamerCYT/rtos-freertos-esp32s3

TP. Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

Tóm tắt

Báo cáo trình bày một chương trình FreeRTOS gồm 8 task ứng dụng và 1 software timer chạy trên kit ESP32-S3 DevKitC-1 (2 nhân, 240 MHz). Sáu task theo đề điều khiển 3 LED, 2 nút nhấn, 1 quang trở và màn hình OLED; hai task bổ sung làm gatekeeper cho cổng Serial và giám sát hệ thống qua ngắt nút BOOT. Các task trao đổi với nhau bằng binary semaphore, mutex, queue, event group, task notification và critical section.

Chương trình được build bằng PlatformIO trên Windows và nạp vào kit thật. Log Serial và video quay mạch xác nhận: LED3 sáng đúng **3 000 ms** sau mỗi lần nhấn BTN2 (7/7 lần đo trong hai buổi chạy), heartbeat đúng chu kỳ **5 000 ms**, hàng đợi log không rơi tin nào (`drop=0`). Khi che tay lên quang trở, độ sáng đo được giảm từ 57 % (~52 lux) xuống 16 % (~3 lux) trong vòng 1–2 giây và trở lại ngay khi bỏ tay ra. Báo cáo cũng ghi lại quá trình tìm lỗi bus I2C (chân SDA bị chập GND) và một lỗi chưa giải quyết: bộ đếm toàn cục đôi khi bị đặt lại về giá trị nhỏ.

8 + 1	3,000 ms	57 → 16 %	7.1 %	11.5 %
task + software timer	LED3 sáng, 7/7 lần đo	che tay lên quang trở	RAM (23 184 B)	Flash (384 667 B)

Toàn bộ mã nguồn, firmware build sẵn và video chạy thử ở kho GitHub (Phụ lục A). Báo cáo không chép code; mỗi kỹ thuật được chỉ rõ tên hàm để tra trong `src/main.cpp`.

1. Yêu cầu đề tài

Đề yêu cầu viết chương trình FreeRTOS trên ESP32, mỗi chức năng là một task độc lập, giao tiếp và đồng bộ qua cơ chế của kernel:

Bảng 1. Sáu task bắt buộc

Task	Yêu cầu của đề
1	LED1 nháy với chu kỳ T = 200 ms
2	Đọc cảm biến ánh sáng, lưu vào biến toàn cục, 500 ms/lần
3	Button1 đảo trạng thái LED2
4	Nhấn Button2 thì cấp (give) binary semaphore
5	Chờ semaphore; khi nhận được thì bật LED3 trong 3 s rồi tắt, chờ tiếp
6	Hiển thị trạng thái LED2, giá trị ánh sáng và LED3 (đếm ngược tới OFF) lên OLED, 500 ms/lần

Ngoài phần bắt buộc, bài làm áp dụng thêm 8 kỹ thuật FreeRTOS (Bảng 4): mutex, queue với gatekeeper task, event group, ngắt phần cứng kết hợp task notification, critical section đa nhân, software timer, giám sát stack/heap và xoá task.

2. Cơ sở lý thuyết FreeRTOS

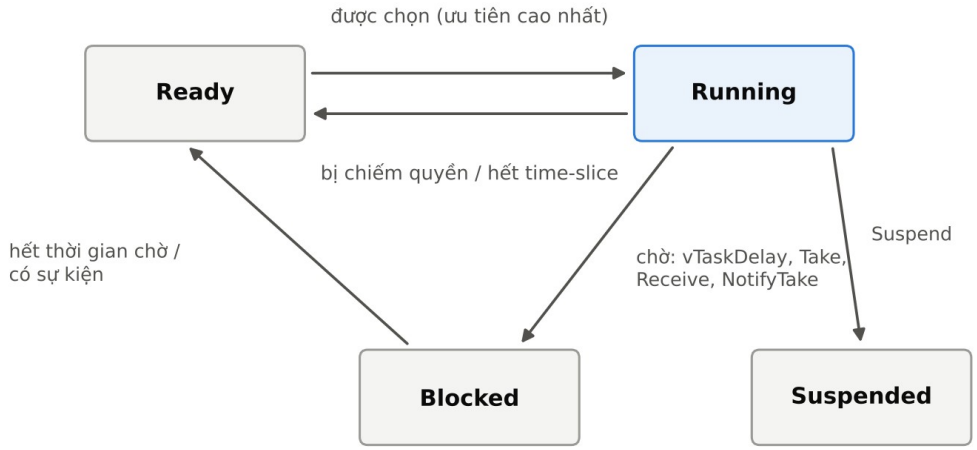
2.1. Kernel và bộ lập lịch

FreeRTOS là kernel thời gian thực mã nguồn mở (MIT), viết bằng C. Thay cho một vòng `loop()` tuần tự, chương trình được chia thành nhiều task; bộ lập lịch chọn task chạy theo độ ưu tiên. ESP-IDF dùng bản FreeRTOS đa nhân (SMP): mỗi nhân có bộ lập lịch riêng, task có thể ghim vào core 0 hoặc core 1.

- **Preemptive theo độ ưu tiên:** task Ready có priority cao nhất luôn được chạy, và chiếm CPU ngay khi được đánh thức.
- **Time slicing:** các task cùng priority chia lượt theo từng tick (ESP32: 1 tick = 1 ms).
- **Idle task:** priority 0, chạy khi không có task nào Ready, đồng thời giải phóng bộ nhớ của task đã xoá.

2.2. Trạng thái của task

Mỗi task luôn ở một trong bốn trạng thái (Hình 1). Task chỉ tốn CPU khi Running; khi chờ thời gian hoặc sự kiện, nó nằm ở Blocked. Thiết kế tốt là để hầu hết task nằm ở Blocked phần lớn thời gian.



Hình 1. Máy trạng thái của task trong FreeRTOS. Mũi tên ghi tên API gây ra chuyển trạng thái; trong bài, các task chu kỳ chuyển Running → Blocked bằng `vTaskDelayUntil`, Task 5 bằng `xSemaphoreTake`.

2.3. Cơ chế giao tiếp và đồng bộ

Bảng 2. Các cơ chế của FreeRTOS dùng trong bài

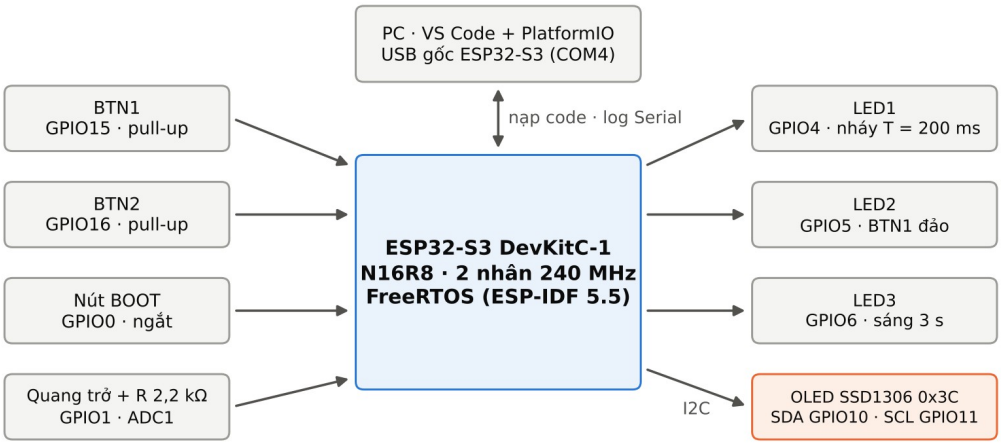
Cơ chế	Dùng khi	Đặc điểm chính
Queue	Truyền dữ liệu giữa task	Sao chép theo giá trị, FIFO, có timeout
Binary semaphore	Báo hiệu sự kiện	Chỉ 0 hoặc 1 token; give nhiều lần vẫn là 1
Mutex	Bảo vệ tài nguyên dùng chung	Có kế thừa ưu tiên; task nào take thì task đó give
Event group	Chờ một hoặc nhiều điều kiện	Chờ theo AND/OR trên các bit cờ
Task notification	Báo hiệu tới một task cụ thể	Không cần tạo đối tượng, nhẹ hơn semaphore
Software timer	Gọi hàm định kỳ	Callback chạy trong timer daemon task, không được block
Critical section	Đoạn code rất ngắn cần nguyên tử	Trên ESP32 là spinlock portMUX vì có 2 nhân

Trong hàm phục vụ ngắt (ISR) chỉ được gọi API có hậu tố `FromISR`, vì ISR không được phép block; nếu ISR vừa đánh thức task ưu tiên cao hơn thì gọi `portYIELD_FROM_ISR` để chuyển ngữ cảnh ngay khi thoát ngắt.

3. Thiết kế hệ thống

3.1. Phần cứng

Hình 2 là sơ đồ khối mạch. Tất cả linh kiện dùng nguồn 3,3 V của kit; nút nhấn nối GND và dùng điện trở kéo lên bên trong chip nên nhấn là mức LOW.

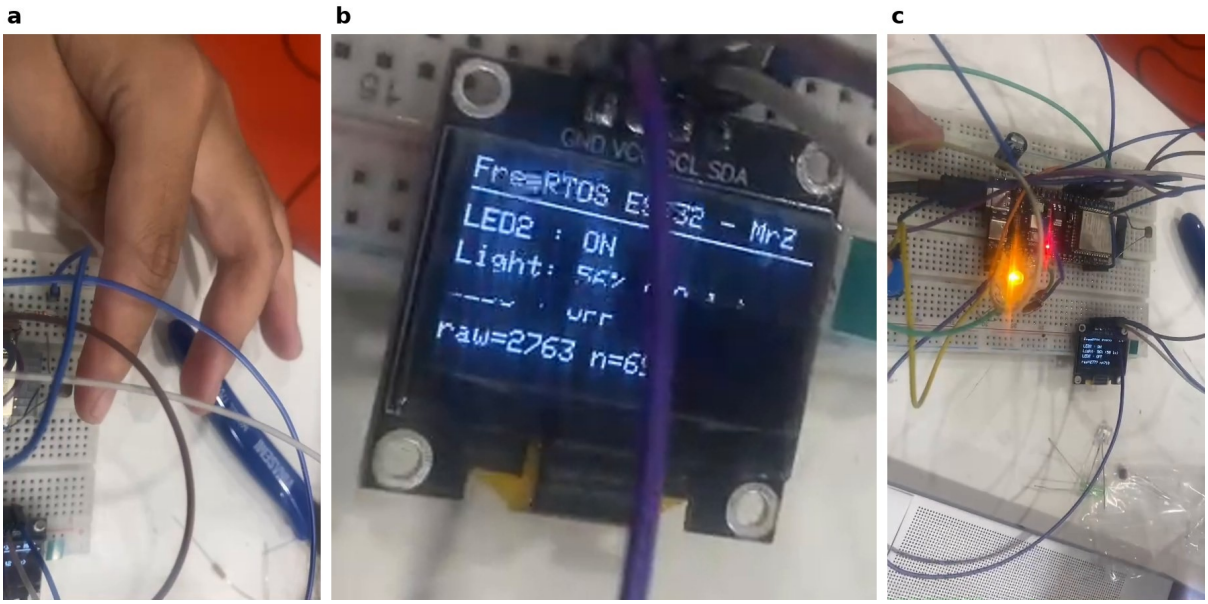


Hình 2. Sơ đồ khối phần cứng trên ESP32-S3 DevKitC-1. OLED (màu cam) được chuyển sang GPIO10/11 sau khi phát hiện chân GPIO8 bị kéo xuống GND trên kit (mục 6.1).

Bảng 3. Gán chân (khai báo trong `include/config.h`)

Linh kiện	Chân ESP32-S3	Ghi chú
LED1, LED2, LED3 + điện trở 220 Ω	GPIO4, 5, 6	Output
BTN1, BTN2 (nút 4 chân)	GPIO15, 16	INPUT_PULLUP, nhấn = LOW
Nút BOOT có sẵn trên kit	GPIO0	Ngắt cạnh xuống cho Task 8
Quang trở 5 mm + điện trở 2,2 kΩ	GPIO1 (ADC1_CH0)	Cầu chia áp, ADC 12 bit, suy hao 11 dB
OLED SSD1306 0,96" 128×64	SDA GPIO10, SCL GPIO11	I2C 100 kHz, địa chỉ 0x3C

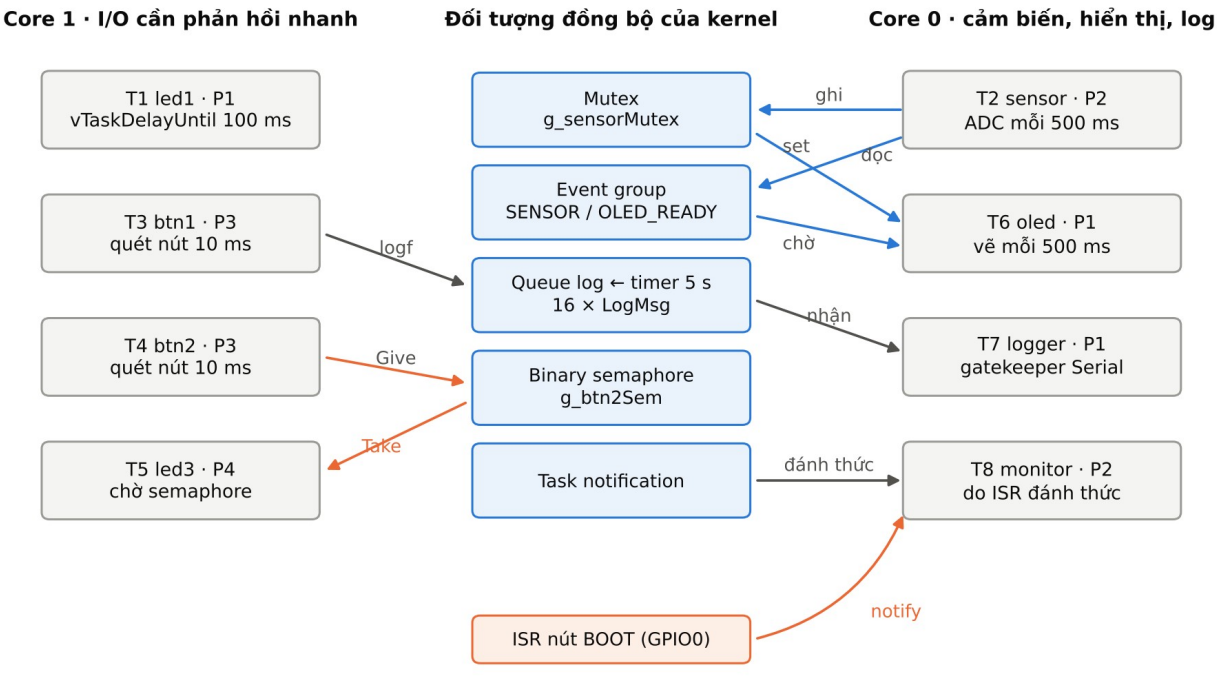
Quang trở được mắc thành cầu chia áp: 3V3 → điện trở 2,2 kΩ → GPIO1 → quang trở → GND. Trời càng tối, điện trở của quang trở càng lớn nên điện áp ở GPIO1 càng gần 3,3 V; trời sáng thì điện áp giảm. Cách đổi điện áp ra độ sáng trình bày ở mục 4.8. Cấu hình cho ESP32 thường (DevKit V1) cũng có sẵn trong cùng file, chọn tự động theo chip.



Hình 3. Mạch thật, cắt từ video chạy thử ngày 13/11. (a) Che tay lên quang trở để thử Task 2. (b) Phóng to OLED: LED2 ON, độ sáng 56 %, LED3 OFF, raw = 2763 (trùng với dòng log ở giây 346). (c) LED2 sáng sau khi nhấn BTN1, OLED cập nhật theo.

3.2. Kiến trúc phần mềm

Hình 4 cho thấy cách 8 task được chia trên 2 nhân và các đối tượng đồng bộ nối giữa chúng. Nguyên tắc chia: task cần phản hồi nhanh với người dùng (LED, nút nhấn) ở core 1; task đọc cảm biến, hiển thị và log ở core 0.



Hình 4. Kiến trúc phần mềm. Mỗi ô là một task (tên, priority, chu kỳ); ô bo tròn ở giữa là đối tượng của kernel. Mũi tên liền: dữ liệu; mũi tên đứt: báo hiệu.

Bảng 4. Phân bổ task

Task	Core	Prio	Kích hoạt	Giao tiếp
1 led1	1	1	vTaskDelayUntil 100 ms	Không

Task	Core	Prio	Kích hoạt	Giao tiếp
2 sensor	0	2	vTaskDelayUntil 500 ms	Ghi g_sensor (mutex), set EVT_SENSOR_READY
3 btn1	1	3	Quét 10 ms	Đảo LED2, gửi log
4 btn2	1	3	Quét 10 ms	Give g_btn2Sem
5 led3	1	4	Take g_btn2Sem	Ghi hạn tắt LED3 (critical section)
6 oled	0	1	vTaskDelayUntil 500 ms	Đọc g_sensor (mutex), chờ event group
7 logger	0	1	xQueueReceive	Task duy nhất được ghi Serial
8 monitor	0	2	ulTaskNotifyTake	Được ISR nút BOOT đánh thức
heartbeat	Không	Không	Software timer 5 s	Gửi log trạng thái

Lý do chọn priority: Task 5 cao nhất (4) để LED3 bật ngay khi có semaphore; hai task nút nhấn (3) cao hơn task hiển thị để không bỏ sót lần nhấn; OLED và logger thấp nhất (1) vì chậm vài chục mili-giây cũng không ảnh hưởng người dùng.

4. Hiện thực

Bảng 5 đối chiếu mỗi kỹ thuật với nơi nó xuất hiện trong mã nguồn. Phần dưới bảng giải thích những quyết định thiết kế không thấy được khi chỉ đọc code.

Bảng 5. Ánh xạ kỹ thuật FreeRTOS sang mã nguồn

Mã	Kỹ thuật	Hàm FreeRTOS	Vị trí trong main.cpp
K1	Tạo task, priority, stack, ghim nhân	xTaskCreatePinnedToCore	setup()
K2	Chu kỳ không trôi	vTaskDelayUntil, pdMS_TO_TICKS	task1, task2, task6
K3	Nhường CPU khi chờ	vTaskDelay	task3, task4, task8
K4	Binary semaphore	xSemaphoreGive / Take	task4 → task5
K5	Mutex	xSemaphoreCreateMutex / Take / Give	task2, task6
K6	Queue + gatekeeper	xQueueSend / Receive	logf(), task7
K7	Event group	xEventGroupSetBits / WaitBits	task2 → task6
K8	Ngắt + task notification	vTaskNotifyGiveFromISR, ulTaskNotifyTake	isr_boot → task8
K9	Critical section 2 nhân	portENTER_CRITICAL / EXIT	task4, task5, task6, logf
K10	Software timer	xTimerCreate / Start	heartbeat_cb
K11	Giám sát stack, heap	uxTaskGetStackHighWaterMark	task8
K12	Xoá task	vTaskDelete(NULL)	loop()

4.1. Chu kỳ chính xác cho LED1 (K2)

Chu kỳ T = 200 ms nghĩa là đảo LED mỗi 100 ms. `vTaskDelay(n)` đếm từ lúc gọi nên chu kỳ thật bằng n cộng thời gian chạy thân vòng lặp, và trôi dần. `vTaskDelayUntil` chờ tới mốc tuyệt đối `last + n` rồi cập nhật mốc, nên chu kỳ không phụ thuộc thời gian xử lý. Task 2 và Task 6 dùng cùng cách.

4.2. Nút nhấn: chống dội và nhường CPU (K3)

Task 3 và 4 quét nút mỗi 10 ms. Hàm `pressed_edge` chỉ báo một lần ở cạnh nhấn và bỏ qua các lần dội trong 30 ms, dựa trên `xTaskGetTickCount`. Lệnh `vTaskDelay` cuối vòng lặp là bắt buộc: nếu thiếu, task priority 3 chạy liên tục, Task 1 cùng nhân bị "đói" CPU và watchdog của idle task báo lỗi.

4.3. Semaphore giữa Task 4 và Task 5 (K4)

Semaphore được tạo ở trạng thái rỗng nên Task 5 nằm ở Blocked, không tốn CPU. Khi Task 4 give, Task 5 có priority 4 lớn hơn 3 nên chiếm CPU ngay, trước cả khi Task 4 kịp ghi log. Log thật cho thấy đúng điều này: dòng "LED3 ON 3s" của Task 5 luôn xuất hiện trước dòng "BTN2 give sem: OK" của Task 4 với cùng mốc thời gian (Hình 6a). Vì là binary semaphore, nhấn BTN2 nhiều lần trong lúc LED3 đang sáng chỉ tích được tối đa một token.

4.4. Dữ liệu dùng chung giữa hai nhân (K5, K9)

Biến `g_sensor` có 4 trường, được Task 2 ghi trên core 0 và Task 6 đọc. Không có mutex, Task 6 có thể đọc lúc Task 2 mới ghi được một nửa. Task 6 chỉ giữ mutex trong lúc chép cấu trúc sang biến cục bộ; phần vẽ OLED qua I2C (vài mili-giây) nằm ngoài vùng găng để Task 2 không phải chờ.

Hai biến hạn tắt LED3 và trạng thái LED3 phải đổi cùng lúc, nên dùng critical section thay vì mutex. Trên ESP32, critical section là spinlock: tắt ngắt trên nhân hiện tại và chặn nhân còn lại. Vì vậy vùng găng chỉ chứa vài phép gán, không gọi hàm nào có thể block.

4.5. Gatekeeper cho cổng Serial (K6)

Nhiều task cùng in ra Serial sẽ làm chữ xen lẫn nhau. Chỉ Task 7 được dùng Serial; các task khác gọi `logf()`, hàm này định dạng chuỗi rồi gửi vào queue với timeout 0, nên không bao giờ làm task gọi bị block. Nếu queue đầy, tin bị bỏ và bộ đếm `drop` tăng; heartbeat in bộ đếm này mỗi 5 s để phát hiện nghẽn.

4.6. Event group, ngắt và software timer (K7, K8, K10)

- **Event group:** Task 6 chờ bit `EVT_SENSOR_READY` trước khi vẽ, nên OLED không hiện số rác lúc khởi động. Bit thứ hai, `EVT_OLED_READY`, cho biết đã tìm thấy màn hình.
- **Ngắt nút BOOT:** ISR nằm trong IRAM và chỉ gửi notification tới Task 8. Việc nặng (duyệt danh sách task, đọc stack) do Task 8 làm ở ngữ cảnh task — mô hình deferred interrupt processing.
- **Software timer:** callback heartbeat chạy trong timer daemon task mỗi 5 s, chỉ đọc hai bộ đếm trong critical section rồi gửi log.
- **Xoá task:** mọi việc đã có task riêng nên `loop()` gọi `vTaskDelete(NULL)` để tự xoá `loopTask` của Arduino.

4.7. Tự dò chân I2C cho OLED

Sau sự cố ở mục 6.1, hàm `oled_init` được viết lại để thử lần lượt nhiều cặp chân SDA/SCL. Với mỗi cặp, chương trình đọc mức logic khi đã bật kéo lên: chân nào đọc 0 là đang bị chập GND, bỏ qua ngay. Cặp nào có thiết bị trả lời ở 0x3C hoặc 0x3D thì dùng. Trong lúc chạy, Task 6 kiểm tra ACK sau mỗi khung hình; mất màn hình 6 lần liên tiếp thì chuyển sang in trạng thái ra Serial và thử khởi tạo lại sau mỗi 10 khung hình, nên rút dây OLED ra cắm lại không cần reset kit.

4.8. Đổi điện áp quang trở ra độ sáng

Task 2 đọc điện áp bằng `analogReadMilliVolts`, hàm này đã hiệu chỉnh ADC theo dữ liệu eFuse của từng chip nên chính xác hơn đọc số raw rồi tự nhân. Từ điện áp V ở GPIO1, điện trở quang trở là R =

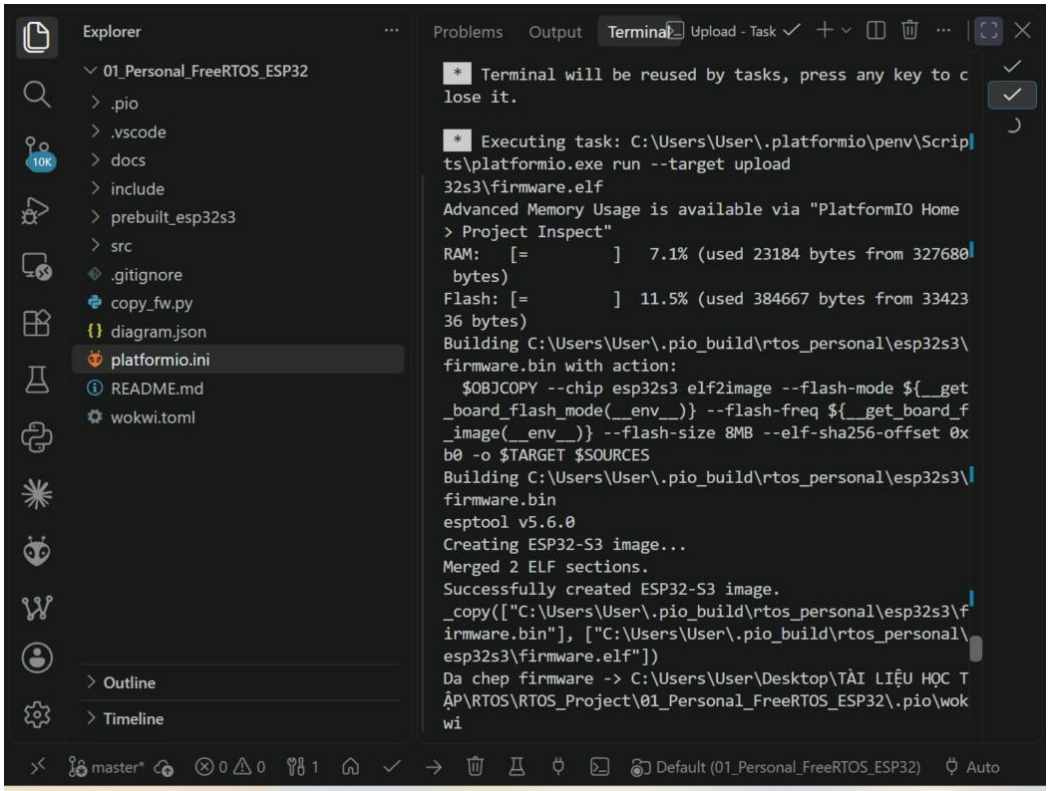
2200 · V / (3,3 − V). Quang trở 5 mm loại GL55xx có R ≈ 15 kΩ ở 10 lux và giảm theo luỹ thừa với độ rọi (hệ số 0,7), từ đó suy ra lux. Con số lux chỉ là ước lượng vì mỗi quang trở lệch nhau khá nhiều; để so sánh sáng/tối thì đủ dùng.

Phần trăm hiển thị trên OLED dùng thang log: 1 lux là 0 %, 1 000 lux là 100 %. Lý do: mắt người cảm nhận độ sáng theo tỉ lệ, phòng 50 lux và phòng 500 lux khác nhau rõ, trong khi thang tuyến tính theo điện áp sẽ dồn hết vùng ánh sáng trong nhà vào vài phần trăm. Với điện trở 2,2 kΩ, phòng học ~50 lux cho khoảng 57 %, che tay còn dưới 20 %, rọi đèn lên gần 100 %. Hằng số `LDR_R_FIXED`, `LDR_RL10`, `LDR_GAMMA` nằm trong `config.h`, đổi điện trở khác chỉ cần sửa một dòng.

5. Kết quả chạy trên kit

5.1. Build và nạp

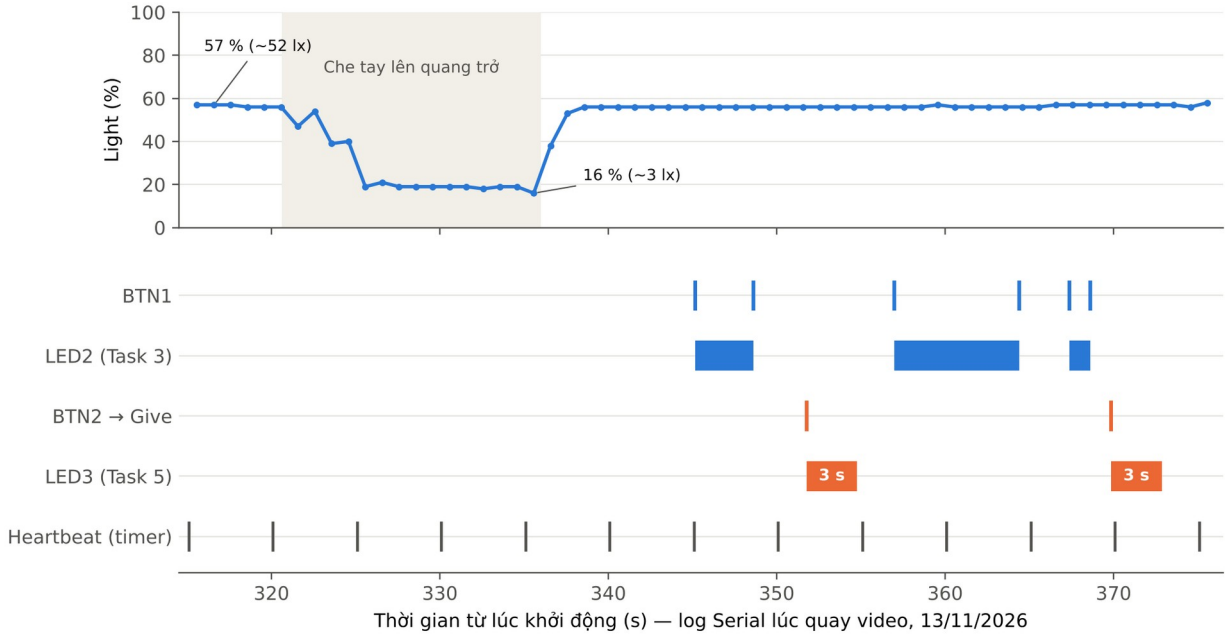
Môi trường: VS Code, PlatformIO Core 6.2.0, platform pioarduino 55.03.30 (Arduino-ESP32 3.3.0, ESP-IDF 5.5), thư viện Adafruit SSD1306 và GFX. Build thành công, 0 lỗi; esptool nhận chip ESP32-S3 (QFN56) rev 0.2, PSRAM 8 MB, flash 16 MB, ghi và kiểm tra hash firmware thành công.



Hình 5. Build trong VS Code + PlatformIO: RAM 23 184 / 327 680 B (7,1 %), Flash 384 667 / 3 342 336 B (11,5 %).

5.2. Đáp ứng thời gian

Hình 6 dựng lại từ log Serial trong 22 giây chạy thật. Che tay lên quang trở, độ sáng tụt từ 57 % xuống 16–19 % rồi trở lại 56 % ngay khi bỏ tay. Mỗi lần nhấn BTN2, LED3 sáng đúng 3 000 ms; nhấn BTN1, LED2 đổi trạng thái tức thì (6 lần); heartbeat cách đều 5 000 ms. Các mốc trùng tới từng mili-giây vì đều tính bằng tick của kernel.



Hình 6. Dựng lại từ log Serial trong lúc quay video (giây 315–376 kể từ khởi động). Trên: độ sáng Task 2 đo mỗi giây, vùng xám là lúc che tay. Dưới: thanh màu là thời gian LED sáng, vạch là lần nhấn nút và heartbeat. Hai lần LED3 sáng: 351 772–354 772 và 369 852–372 852 ms, đúng 3 000 ms.

a

```
[  72572 ms][c0] LDR raw=2630 2186mV light=59% ~59 lx
[  73572 ms][c0] LDR raw=2613 2188mV light=59% ~59 lx
[  74572 ms][c0] LDR raw=2632 2179mV light=59% ~60 lx
[  75072 ms][c0] heartbeat up=75s btn2=1 drop=0
[  75572 ms][c0] LDR raw=2587 2149mV light=60% ~64 lx
[  76572 ms][c0] LDR raw=691 590mV light=100% ~1370 lx
[  77572 ms][c0] LDR raw=620 536mV light=100% ~1617 lx
[  78572 ms][c0] LDR raw=746 639mV light=100% ~1191 lx
[  79572 ms][c0] LDR raw=2450 2045mV light=62% ~77 lx
[  80072 ms][c0] heartbeat up=80s btn2=1 drop=0
[  80572 ms][c0] LDR raw=2628 2179mV light=59% ~60 lx
[  81572 ms][c0] LDR raw=2281 1886mV light=67% ~103 lx
[  82572 ms][c0] LDR raw=1013 858mV light=94% ~692 lx
[  83572 ms][c0] LDR raw=2516 2093mV light=61% ~71 lx
[  84572 ms][c0] LDR raw=2677 2224mV light=58% ~55 lx
[  85072 ms][c0] heartbeat up=85s btn2=1 drop=0
[  85572 ms][c0] LDR raw=2683 2228mV light=57% ~55 lx
[  85872 ms][c0] BTN1 -> LED2 ON
[  86072 ms][c0] BTN1 -> LED2 OFF
[  86442 ms][c0] BTN1 -> LED2 ON
[  86572 ms][c0] LDR raw=2683 2231mV light=57% ~54 lx
[  86882 ms][c0] BTN1 -> LED2 OFF
[  87042 ms][c0] BTN1 -> LED2 ON
[  87572 ms][c0] LDR raw=2671 2222mV light=58% ~55 lx
[  87682 ms][c0] BTN1 -> LED2 OFF
[  88572 ms][c0] LDR raw=2665 2215mV light=58% ~56 lx
[  88642 ms][c0] LED3 ON 3s
[  88642 ms][c0] BTN2 give sem: OK
[  89572 ms][c0] LDR raw=2663 2213mV light=58% ~56 lx
[  89662 ms][c0] BTN1 -> LED2 ON
[  90012 ms][c0] BTN1 -> LED2 OFF
[  90072 ms][c0] heartbeat up=90s btn2=2 drop=0
[  90202 ms][c0] BTN1 -> LED2 ON
[  90572 ms][c0] LDR raw=2663 2215mV light=58% ~56 lx
[  91572 ms][c0] LDR raw=2661 2210mV light=58% ~57 lx
[  91642 ms][c0] LED3 OFF, cho tiep
```

b

```
[ 158612 ms][c0] BTN2 give sem: da co san
[ 159202 ms][c0] BTN2 give sem: da co san
[ 159572 ms][c0] LDR raw=2710 2249mV light=57% ~52 lx
[ 160012 ms][c0] LED3 OFF, cho tiep
[ 160012 ms][c0] LED3 ON 3s
[ 160072 ms][c0] heartbeat up=160s btn2=9 drop=0
[ 160112 ms][c0] BTN2 give sem: OK
[ 160482 ms][c0] BTN2 give sem: da co san
[ 160572 ms][c0] LDR raw=2713 2249mV light=57% ~52 lx
[ 160832 ms][c0] BTN2 give sem: da co san
[ 161162 ms][c0] BTN2 give sem: da co san
[ 161572 ms][c0] LDR raw=2708 2226mV light=57% ~55 lx
[ 162572 ms][c0] LDR raw=2703 2243mV light=57% ~53 lx
[ 163272 ms][c0] BTN2 give sem: OK
[ 163572 ms][c0] LDR raw=2696 2223mV light=58% ~55 lx
[ 163882 ms][c0] BTN2 give sem: da co san
[ 164572 ms][c0] LDR raw=2696 2238mV light=57% ~54 lx
[ 165072 ms][c0] heartbeat up=165s btn2=15 drop=0
[ 165572 ms][c0] LDR raw=2703 2247mV light=57% ~53 lx
[ 166012 ms][c0] LED3 OFF, cho tiep
[ 166012 ms][c0] LED3 ON 3s
[ 166572 ms][c0] LDR raw=2697 2238mV light=57% ~54 lx
[ 167364 ms][c0] === MONITOR: 14 tasks, heap 318552 B ===
[ 167364 ms][c0] led1     prio=1 stackFree=1328 B
[ 167364 ms][c0] sensor   prio=2 stackFree=928 B
[ 167364 ms][c0] btn1     prio=3 stackFree=1472 B
[ 167364 ms][c0] btn2     prio=3 stackFree=1484 B
[ 167364 ms][c0] led3     prio=4 stackFree=1480 B
[ 167364 ms][c0] oled     prio=1 stackFree=1856 B
[ 167364 ms][c0] logger   prio=1 stackFree=1336 B
[ 167364 ms][c0] monitor  prio=2 stackFree=2464 B
[ 167572 ms][c0] LDR raw=2709 2251mV light=57% ~52 lx
[ 168572 ms][c0] LDR raw=2709 2247mV light=57% ~53 lx
[ 169012 ms][c0] LED3 OFF, cho tiep
[ 169572 ms][c0] LDR raw=2711 2251mV light=57% ~52 lx
[ 170072 ms][c0] heartbeat up=170s btn2=15 drop=0
```

Hình 7. Log Serial ngày 13/11. (a) Rọi đèn vào quang trở: điện áp xuống 536 mV, độ sáng 100 % (~1 617 lux); sau đó nhấn BTN2 rồi nhấn BTN1 ba lần trong lúc LED3 đang sáng, LED2 vẫn đổi ngay. (b) Bấm BTN2 liên tục: lần đầu `OK`, các lần sau `da co san`; LED3 tắt và bật lại cùng một mốc thời gian; nhấn BOOT in bảng 14 task, `drop=0`.

Bảng 6. Số đo từ log Serial

Đại lượng	Yêu cầu	Đo được	Số mẫu
Thời gian LED3 sáng	3 s	3 000 ms (sai số 0 ms)	7 (2 buổi)

Đại lượng	Yêu cầu	Đo được	Số mẫu
Độ sáng khi che tay	Giảm rõ	57 % → 16 % (52 → 3 lux)	1
Chu kỳ heartbeat	5 s	5 000 ms	> 120
Log bị rơi	Không yêu cầu	0	Hơn 10 phút
BTN1 → LED2 đổi	Đảo trạng thái	Đúng ở mọi lần nhấn	4
Heap còn trống	Không yêu cầu	318 552 B	1

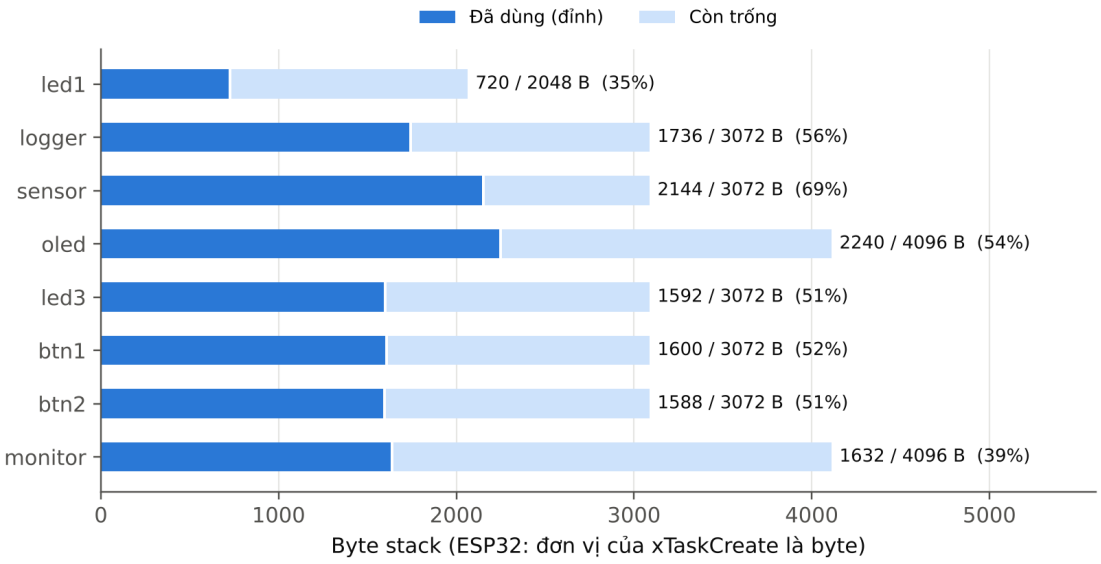
5.3. Giám sát task và stack

Nhấn nút BOOT làm Task 8 in bảng trạng thái (Hình 8): 14 task trong hệ thống (8 task của bài, 1 timer daemon, còn lại là task của ESP-IDF như idle ×2, IPC ×2, esp_timer), heap trống 318 552 byte.

```
[ 167364 ms][c0] === MONITOR: 14 tasks, heap 318552 B ===
[ 167364 ms][c0] led1     prio=1 stackFree=1328 B
[ 167364 ms][c0] sensor   prio=2 stackFree=928 B
[ 167364 ms][c0] btn1     prio=3 stackFree=1472 B
[ 167364 ms][c0] btn2     prio=3 stackFree=1484 B
[ 167364 ms][c0] led3     prio=4 stackFree=1480 B
[ 167364 ms][c0] oled     prio=1 stackFree=1856 B
[ 167364 ms][c0] logger   prio=1 stackFree=1336 B
[ 167364 ms][c0] monitor  prio=2 stackFree=2464 B
```

Hình 8. Bảng do Task 8 in khi nhấn BOOT (ngày 13/11): priority và stack chưa dùng tới (high-water mark) của từng task.

Hình 9 so stack cấp phát với phần đã dùng. Task dùng nhiều nhất là sensor (69 %): từ khi in thêm dòng log độ sáng mỗi giây, hàm định dạng số thực `vsnprintf` chiếm khá nhiều stack. Mức này vẫn an toàn nhưng nếu thêm việc cho Task 2 thì nên tăng stack lên 4 096 byte. Task LED1 mới dùng 34 %, có thể giảm để tiết kiệm RAM.



Hình 9. Stack cấp phát và phần đã dùng của từng task, tính từ high-water mark ở hình trước.

5.4. Kiểm thử chức năng

Bảng 7. Kiểm thử chức năng

#	Thao tác	Kỳ vọng	Kết quả
1	Cấp nguồn	LED1 nháy 5 lần/s; log “Khoi dong xong”	Đạt (video, log)
2	Che tay lên quang trở	Light giảm rõ, trở lại khi bỏ tay	Đạt: 57 % → 16 % (52 → 3 lux)
2b	Rọi đèn vào quang trở	Light tăng gần 100 %	Đạt: 100 % (~1 600 lux)
3	Nhấn BTN1	LED2 đảo, OLED cập nhật	Đạt
4	Nhấn BTN2	LED3 sáng 3 s, OLED đếm ngược rồi OFF	Đạt, 3 000 ms
5	Nhấn BTN2 3 lần liên tiếp	Lần 2, 3 báo “da co san”	Đạt: 1 lần OK, các lần sau “da co san”; LED3 sáng thêm đúng 1 lượt
6	Nhấn BTN1 khi LED3 đang sáng	LED2 vẫn đổi ngay	Đạt: LED2 đổi 3 lần trong lúc LED3 sáng
7	Nhấn BOOT	In bảng task, heap, stack	Đạt
8	Để chạy lâu	Heartbeat đều, drop = 0	Đạt, hơn 10 phút
9	Rút/cắm lại OLED	Tự phát hiện và khởi tạo lại	Đạt

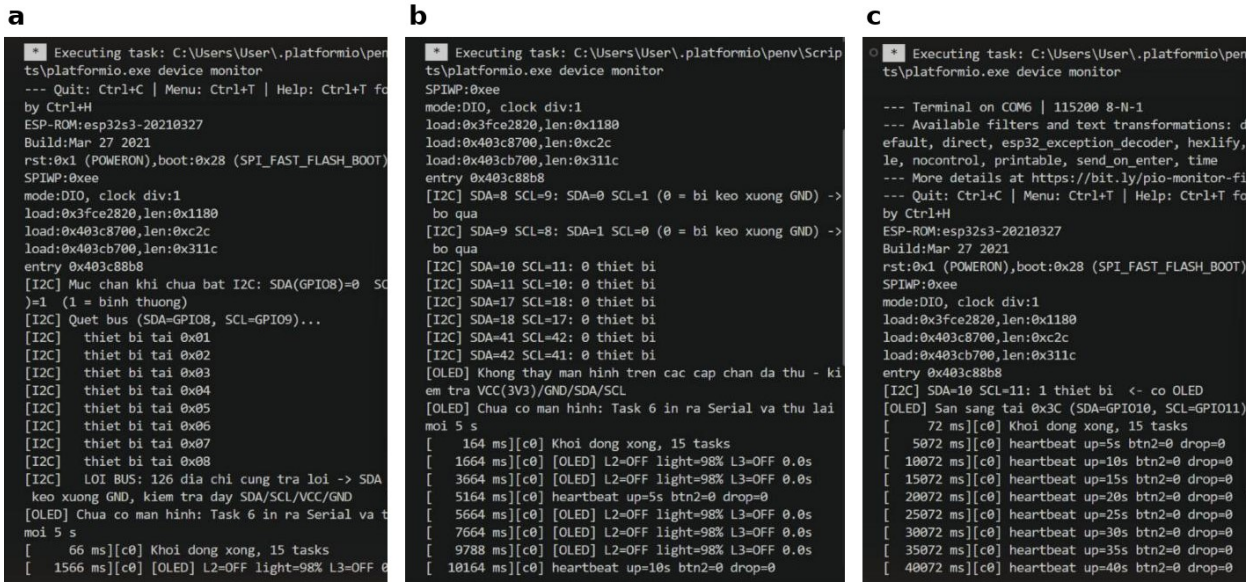
Tất cả kịch bản đều đạt. Kịch bản 5 cho thấy rõ tính chất của binary semaphore: bấm BTN2 nhiều lần trong lúc LED3 đang sáng thì chỉ lần đầu cất được token, các lần sau bị bỏ, nên LED3 chỉ sáng nối thêm một lượt 3 s (log: LED3 OFF và LED3 ON cùng mốc 160 012 ms). Nếu muốn mỗi lần bấm đều được tính thì dùng counting semaphore hoặc queue.

6. Quá trình gỡ lỗi

6.1. OLED không hiển thị: chân SDA bị chập GND

Ban đầu OLED nối SDA = GPIO8, SCL = GPIO9 và không hiển thị. Hình 10 tóm tắt ba bước tìm ra nguyên nhân.

1. **Quét bus I2C:** cả 126 địa chỉ đều “trả lời”. Một bus bình thường chỉ có 1–2 thiết bị trả lời, nên đây là dấu hiệu đường SDA luôn ở mức 0 (bộ quét hiểu mức 0 là ACK).
2. **Đọc mức chân khi chưa bật I2C:** với điện trở kéo lên bật, GPIO8 vẫn đọc 0 còn GPIO9 đọc 1. Kết luận: GPIO8 bị kéo xuống GND trên phần cứng (dây cắm hoặc board), không phải lỗi thư viện hay địa chỉ.
3. **Chuyển chân:** viết hàm tự dò cặp chân (mục 4.7) và chuyển OLED sang GPIO10/11. Màn hình được tìm thấy ở 0x3C; đặt xung nhịp 100 kHz và kiểm tra ACK mỗi khung hình, từ đó không còn lỗi I2C.



Hình 10. Ba bước gỡ lỗi I2C qua log Serial. (a) Mọi địa chỉ đều trả lời và SDA(GPIO8) = 0. (b) Bộ tự dò bỏ qua cặp 8/9, thử các cặp khác. (c) Sau khi chuyển dây: tìm thấy OLED ở 0x3C trên GPIO10/11, khởi động sạch.

6.2. Lỗi môi trường build trên Windows

- **Đường dẫn có dấu tiếng Việt** làm trình liên kết `ld` báo "Invalid argument". Cách sửa: đặt thư mục build ra `C:\Users\<tên>\.pio_build`, thêm script `copy_fw.py` chép firmware về lại thư mục dự án cho Wokwi.
- **PlatformIO Core 6.2.0** không đọc được platform 55.03.312, nên dùng bản 55.03.30.
- **Không có log Serial:** khi cắm cổng USB gốc của chip phải bật `ARDUINO_USB_CDC_ON_BOOT=1`; khi cắm cổng UART (CH343) thì đặt về 0. Lưu ý: với cổng USB gốc, mỗi lần chip reset thì cổng COM biến mất vài giây và Serial Monitor phải mở lại.
- **Log bị mất chữ qua USB:** khi in nhiều dòng liền nhau (bảng BOOT), cổng USB CDC làm rơi một số ký tự giữa dòng. Sửa bằng cách tăng bộ đệm gửi lên 2 048 byte và cho task logger chờ gửi xong từng dòng (`Serial.flush`). Sau khi sửa, bảng in ra đầy đủ (Hình 8).
- **Nạp lỗi vì nhiều cổng COM ảo:** máy cài phần mềm tạo cổng COM ảo (ELTIMA) nên PlatformIO tự dò nhầm cổng. Cách sửa: ghi cố định `upload_port = COM4` và `monitor_port = COM4` trong `platformio.ini`.

6.3. Lỗi còn tồn tại: bộ đếm bị đặt lại

Trong lần chạy dài, bộ đếm số lần nhấn BTN2 trong heartbeat giảm từ 1 về 0 ở khoảng giây thứ 765, trong khi thời gian chạy vẫn tăng liên tục (không reset chip). Trong video, số thứ tự lần đọc cảm biến trên OLED từ 189 trở về 1 rồi đứng yên, trong khi giá trị raw vẫn đổi. Trong buổi chạy ngày 13/11 (hơn 430 giây, có quay video), bộ đếm tăng đúng 1 → 2 → 3 theo số lần nhấn và không bị đặt lại, nên lỗi không xuất hiện lần nào. Cả hai biến đều là biến toàn cục nằm sát nhau trong RAM (vùng 0x3fc98960–0x3fc98980, ngay sau đối tượng `oled` và các handle).

Giả thuyết đang kiểm tra là có đoạn code ghi tràn sang vùng nhớ này theo chu kỳ. Hướng xử lý: dùng watchpoint phần cứng của ESP32-S3 (`esp_cpu_set_watchpoint`) trên địa chỉ `g_btn2Count` để bắt đúng lệnh ghi, và tạm đổi thứ tự khai báo biến để xem lỗi có đi theo địa chỉ hay theo biến. Lỗi này không ảnh hưởng tới các chức năng của đề (LED, nút, semaphore vẫn đúng), nhưng được ghi lại để không trình bày kết quả tốt hơn thực tế.

7. Kết luận và hướng phát triển

Chương trình đáp ứng 6 task của đề trên kit thật và dùng thêm 8 kỹ thuật FreeRTOS. Bài học chính là chọn đúng cơ chế cho từng loại dữ liệu dùng chung: mutex cho cấu trúc nhiều trường, critical section cho vài biến nhỏ cần đổi cùng lúc, queue khi cần truyền dữ liệu, semaphore hoặc notification khi chỉ cần báo hiệu. Việc gỡ lỗi I2C cho thấy log có cấu trúc và kiểm tra mức chân là cách nhanh nhất để tách lỗi phần cứng khỏi lỗi phần mềm.

- Tìm nguyên nhân lỗi đặt lại bộ đếm bằng watchpoint (mục 6.3).
- Thay quét nút bằng ngắt GPIO + queue sự kiện; dùng counting semaphore để không mất lần nhấn.
- Bật `configGENERATE_RUN_TIME_STATS` để đo phần trăm CPU của từng task.

Tài liệu tham khảo

1. FreeRTOS Kernel Developer Docs, https://www.freertos.org/Documentation
2. Espressif, ESP-IDF Programming Guide: FreeRTOS (SMP), https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/freertos.html
3. R. Barry, Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide.
4. Adafruit, SSD1306 OLED library, https://github.com/adafruit/Adafruit_SSD1306

Phụ lục A. Mã nguồn, firmware và video

Toàn bộ dự án ở kho GitHub:

https://github.com/JackGamerCYT/rtos-freertos-esp32s3

Bảng 8. Cấu trúc kho mã nguồn

Đường dẫn trong kho	Nội dung
01_Personal_FreeRTOS_ESP32/src/main.cpp	8 task, ISR, timer, logger
01_Personal_FreeRTOS_ESP32/include/config.h	Gán chân, chu kỳ, priority, stack
01_Personal_FreeRTOS_ESP32/platformio.ini	Cấu hình build cho esp32s3 và esp32dev
01_Personal_FreeRTOS_ESP32/prebuilt_esp32s3/	Firmware build sẵn để nạp không cần build
01_Personal_FreeRTOS_ESP32/diagram.json	Mạch mô phỏng Wokwi
media/demo_esp32s3.mp4	Video chạy thử trên kit (57 s, quay ngày 13/11)
03_Reports/	Báo cáo và slide các tuần

Build và nạp: mở thư mục `01_Personal_FreeRTOS_ESP32` bằng VS Code có PlatformIO, chọn env `esp32s3`, bấm Upload rồi Monitor (115200). Hướng dẫn chi tiết trong README của kho.